

Session 14 : Pratique des moteurs — Rigid Bodies & Colliders

Objectifs de la session

- **Installer et configurer** deux moteurs physiques : **Rapier** avec Three.js, et **Jolt** intégré à Godot.
- Comprendre le **setup** de chaque moteur : importmap et WASM côté web, `project.godot` côté Godot.
- Créer **pas à pas** des **rigid bodies** (fixed, dynamic, kinematic) et des **colliders** dans chaque moteur.
- Maîtriser la **boucle de simulation** : `world.step()` + synchronisation manuelle (Rapier) vs `_physics_process` automatique (Godot).
- Démontrer le **CCD** anti-tunneling et le **sleeping** sur la même scène dans les deux moteurs.
- Établir le **tableau comparatif** des API pour passer de l'un à l'autre sans friction.

💡 Le passage à la pratique

La Session 13 a posé le **paysage** et les **concepts** partagés. Cette session est son **pendant pratique** : on met les mains dans le code. Deux moteurs, une même scène — un sol, une pyramide de cubes, un boulet de canon — pour comparer **concrètement** les deux philosophies : la **bibliothèque** (Rapier, qu'on intègre à Three.js) et le **moteur intégré** (Godot + Jolt, où la physique est un nœud de l'arbre).

Partie 1 : Deux philosophies, une même scène

Bibliothèque vs moteur intégré

- **Rapier + Three.js** : la physique est une **bibliothèque externe**. On crée un monde, on crée des corps, on appelle `world.step()`, et on **recopie** manuellement les positions du moteur vers les meshes. La physique et le rendu sont **déconnectés** — c'est nous qui faisons le pont.
- **Godot + Jolt** : la physique est **intégrée au moteur de jeu**. Un `RigidBody3D` est à la fois un corps physique **et** un nœud de scène. Godot gère le `step()` et la synchronisation automatiquement. On n'écrit **aucune** boucle de copie.

💡 Pourquoi comparer les deux ?

Parce qu'en production, le choix se pose. Si vous faites un jeu web avec Three.js, vous **devez** intégrer Rapier. Si vous faites un jeu Godot, la physique est **déjà là**. Comprendre les deux approches vous permet de choisir en connaissance de cause — et de transférer vos connaissances d'un moteur à l'autre, car les **concepts** (rigid body, collider, CCD, sleeping) sont identiques.

Partie 2 : Three.js + Rapier — Fondamentaux & Setup

Documentation & ressources

Liens utiles

- **Rapier (site officiel)** : rapier.rs — documentation, exemples, et le **playground** interactif.
- **Rapier JS (dimforge)** : github.com/dimforge/rapier — les packages `@dimforge/rapier3d-compat` (WASM) et `@dimforge/rapier3d` (Node).
- **Three.js** : threejs.org/docs — la référence de l'API rendu.

- **Exemple du cours** : `examples/session14_rapier.html` + `session14_rapier.js` — la scène complète (sol, pyramide, boulet, HUD sleeping).

Setup — l'importmap

Pourquoi un importmap ?

Les modules ES ne savent pas résoudre les noms nus (`import ... from 'three'`). L'importmap déclare la correspondance entre ces noms et les CDN. Pour Rapier, on ajoute une entrée vers le package `-compat` qui embarque le binaire WASM en base64 :

```
<script type="importmap">
{
  "imports": {
    "three": "https://unpkg.com/three@0.185.1/build/three.module.js",
    "jsm/": "https://unpkg.com/three@0.185.1/examples/jsm/",
    "rapier": "https://cdn.skypack.dev/@dimforge/rapier3d-compat@0.19.3"
  }
}
</script>
```

C'est le **seul** changement côté HTML. Le JS fait ensuite `import RAPIER from 'rapier'` — le nom nu est résolu par l'importmap.

💡 En production (Vite, Webpack)

Pour un vrai projet, on installe le package via npm (`npm install @dimforge/rapier3d-compat`) et l'importmap devient inutile — le bundler résout les noms. L'importmap CDN est pratique pour **prototyper** et pour ce cours : zéro build, un seul fichier à ouvrir.

Le chargement WASM — l'étape asynchrone

Rapier est un binaire, pas du JavaScript

Rapier est écrit en **Rust** et compilé en **WebAssembly** (WASM). Le package `-compat` embarque le binaire en base64. Conséquence : le chargement est **asynchrone** — impossible de créer quoi que ce soit avant `RAPIER.init()`. Toute la fonction d'initialisation est donc `async` :

```
import RAPIER from 'rapier'; // résolu par l'importmap

async function init() {
  await RAPIER.init(); // ← le WASM se décode ici
  // seulement APRÈS : world, bodies, colliders...
}
init().catch(err => console.error(err));
```

⚠ Le piège classique

Si on oublie le `await RAPIER.init()`, toute création de `World` ou de `RigidBody` lève une erreur du type **"RAPIER is not initialized"**. C'est l'équivalent d'oublier d'allumer le moteur avant de conduire.

Le monde physique

L'équivalent de notre « univers » maison

```
const FIXED_DT = 1 / 60;
const world = new RAPIER.World({ x: 0, y: -9.81, z: 0 });
world.timestep = FIXED_DT; // pas fixe – pattern de la Session 5
```

Le monde est le **registre** de tous les corps + la gravité + le pas de temps. C'est l'équivalent de notre tableau `planets` + la constante `G` – mais il gère aussi la `broad phase`, le solveur, le `sleeping`... tout ce qu'on a écrit à la main.

Partie 3 : Three.js + Rapier – Rigid Bodies & Colliders pas à pas

Étape 1 – Le sol : un corps FIXE

RigidBodyDesc.fixed() + ColliderDesc.cuboid()

```
// --- Rendu : le mesh avec la texture grille ---
const mesh = new THREE.Mesh(
  new THREE.BoxGeometry(60, 0.2, 60), gridMaterial);

// --- Physique : corps FIXE (masse infinie, jamais simulé) ---
const body = world.createRigidBody(
  RAPIER.RigidBodyDesc.fixed().setTranslation(0, -0.1, 0));
world.createCollider(
  RAPIER.ColliderDesc.cuboid(30, 0.1, 30) // ← DEMI-tailles !
  .setFriction(params.friction)
  .setRestitution(params.restitution),
  body);
```

Deux objets distincts : le **rigid body** (le comportement – fixe, dynamique...) et le **collider** (la forme – cuboïde, boule...). Un body peut porter **plusieurs** colliders ; un collider sans body n'existe pas.

🗨 Le piège des demi-tailles

`ColliderDesc.cuboid(hx, hy, hz)` prend des **demi-dimensions** (half-extents), alors que `BoxGeometry(w, h, d)` de Three.js prend des dimensions **complètes**. Un cube de 1 m = `BoxGeometry(1,1,1)` côté rendu, `cuboid(0.5, 0.5, 0.5)` côté physique. Se tromper ici double ou réduit la taille du collider – et la physique se comporte bizarrement sans erreur apparente.

Étape 2 – La pyramide : des corps DYNAMIQUES

Le pattern « pairs » – le lien rendu ↔ physique

```
const pairs = []; // { mesh, body } pour CHAQUE objet physique

for (let row = 0; row < 5; row++) {
  for (let i = 0; i < 5 - row; i++) {
    // Rendu
    const mesh = new THREE.Mesh(boxGeo, cubeMaterial);
    scene.add(mesh);
    // Physique : corps DYNAMIQUE
    const body = world.createRigidBody(
```

```

        RAPIER.RigidBodyDesc.dynamic()
            .setTranslation(x, 0.5 + row * 1.05, 0));
world.createCollider(
    RAPIER.ColliderDesc.cuboid(0.5, 0.5, 0.5), body);
// Le lien croisé — c'est LUI qu'on itère dans animate()
pairs.push({ mesh, body });
    }
}

```

Rapier ne sait rien de Three.js, et Three.js ne sait rien de Rapier. Chaque objet physique existe donc **en double** : un mesh (ce qu'on voit) et un body (ce qui est simulé). Le tableau `pairs` maintient la correspondance — c'est le **cœur de toute intégration** moteur physique + moteur de rendu.

Étape 3 — Un corps KINEMATIC (plateforme)

RigidBodyDesc.kinematicPositionBased()

```

// Plateforme mobile : contrôlée par le code, pousse les dynamic bodies
const platform = world.createRigidBody(
    RAPIER.RigidBodyDesc.kinematicPositionBased()
        .setTranslation(0, 1.5, -5));
world.createCollider(
    RAPIER.ColliderDesc.cuboid(3, 0.2, 3), platform);

// Dans la boucle : on déplace la plateforme par le code
function animate() {
    // ...
    const t = performance.now() * 0.001;
    platform.setNextKinematicTranslation({ x: Math.sin(t) * 4, y: 1.5, z: -5 });
    // ...
}

```

Le kinematic **ignore les forces** mais **pousse** les dynamic bodies. On contrôle sa position via `setNextKinematicTranslation()` — la position est appliquée au **prochain** `step()`. C'est l'outil pour les ascenseurs, portes automatiques, plateformes de jeu.

💡 Kinematic vs Dynamic — quand choisir ?

Un ascenseur **Dynamic** rebondirait sous le poids des personnages et oscillerait. Un **Kinematic** suit exactement la trajectoire qu'on lui impose — il pousse les corps qui le touchent sans jamais être dévié. La règle : si le mouvement est **scripté** (sinus, spline, input joueur), c'est kinematic. S'il doit **réagir** aux forces, c'est dynamic.

Étape 4 — La boucle : accumulateur, step, synchronisation

Le fixed timestep de la Session 5, appliqué à Rapier

```

function animate() {
    const dt = Math.min(clock.getDelta(), 0.1);
    accumulator += dt;
    while (accumulator >= FIXED_DT) {
        world.step(); // ← TOUTE la physique ici
        accumulator -= FIXED_DT;
    }
}

```

```

    }
    // - Synchronisation : rendu ← physique -
    for (const { mesh, body } of pairs) {
        const t = body.translation(); // { x, y, z }
        const r = body.rotation(); // quaternion
        mesh.position.set(t.x, t.y, t.z);
        mesh.quaternion.set(r.x, r.y, r.z, r.w);
    }
    renderer.render(scene, camera);
}

```

La physique tourne à **pas fixe** (60 Hz), indépendamment du framerate. Si une frame dure 32 ms, la boucle while fait **deux pas**. Puis on **recopie** les positions et rotations du moteur vers les meshes.

❗ Règle d'or : la physique est la source de vérité

On ne déplace **jamais** le mesh directement — on pousse le body (force, impulsion), et le mesh suit à la frame suivante. Inverser cette relation casse la simulation : le body et le mesh se désynchronisent, les collisions se produisent au mauvais endroit.

Étape 5 — Le boulet : impulsion + CCD

applyImpulse + setCcdEnabled — la démo tunneling

```

function fireCannonball(origin, dir) {
    // 1. Rendu
    const mesh = new THREE.Mesh(
        new THREE.SphereGeometry(0.45, 24, 24),
        new THREE.MeshStandardMaterial({ color: 0x2c3e50, metalness: 0.9 }));
    scene.add(mesh);

    // 2. Physique : CCD activé, collider lourd
    const body = world.createRigidBody(
        RAPIER.RigidBodyDesc.dynamic()
        .setTranslation(origin.x, origin.y, origin.z)
        .setCcdEnabled(params.ccd)); // ← CCD ici !
    world.createCollider(
        RAPIER.ColliderDesc.ball(0.45)
        .setDensity(10), // lourd — 10× la densité par défaut
        body);

    // 3. Impulsion : Session 6 en une ligne
    body.applyImpulse(
        { x: dir.x * params.cannonSpeed, y: dir.y * params.cannonSpeed, z: dir.z *
        params.cannonSpeed },
        true); // true = réveille le corps
    pairs.push({ mesh, body });
}

```

Le boulet est le cas d'école du **tunneling** : à 120 unités/s, il avance de **2 unités par pas** — plus qu'un cube entier. Désactivez le CCD dans la GUI et tirez à haute vitesse : il **traverse** la pyramide. Réactivez-le : il la démolit.

Étape 6 — Le HUD : observer le sleeping

isSleeping() — la preuve que le moteur optimise

```
function updateHUD() {
    let sleeping = 0, dynamic = 0;
    for (const { body } of pairs) {
        if (body.isDynamic()) dynamic++;
        if (body.isSleeping()) sleeping++;
    }
    hud.textContent = `Corps : ${dynamic} | Endormis : ${sleeping}`;
}
```

Laissez la scène se stabiliser : le compteur d'**endormis** grimpe vers le total. Le moteur a **arrêté de simuler** les cubes immobiles — zéro coût CPU pour eux. Touchez la pyramide : les cubes touchés **se réveillent** en cascade.

⚠ Le piège du sleeping

Quand la GUI change la gravité, il faut **réveiller tout le monde** (`body.wakeUp()`) — sinon les corps endormis continuent d'ignorer la nouvelle gravité ! C'est un bug classique : on change un paramètre global et rien ne bouge parce que les objets dorment.

Étape 7 — Le ménage

removeRigidBody + reset

```
function removePair(pair) {
    scene.remove(pair.mesh);
    pair.mesh.geometry.dispose();
    pair.mesh.material.dispose();
    world.removeRigidBody(pair.body); // supprime aussi ses colliders
    pairs = pairs.filter(p => p !== pair);
}
```

Comme toujours : on libère la mémoire GPU (geometry/material) et on retire le body du monde. Les boulets sont plafonnés à 12 pour la performance — un vrai moteur gère des **milliers** de corps, mais chaque `world.step()` a un coût.

Partie 4 : Godot + Jolt — Fondamentaux & Setup

Documentation & ressources

Liens utiles

- **Godot (site officiel)** : godotengine.org — téléchargement, documentation.
- **Godot Physics (docs)** : docs.godotengine.org/.../physics — introduction à la physique 3D.
- **Jolt dans Godot** : github.com/godot-jolt/godot-jolt — le plugin (intégré depuis 4.4).
- **RigidBody3D (API)** : docs.godotengine.org/.../class_rigidbody3d — référence de la classe.
- **Projet du cours** : le dossier `demo-physics` — scène `session13_demo.tscn` + script `session13_demo.gd` (construit la scène au runtime).

Setup — le project.godot

Trois lignes qui comptent

```
[physics]
3d/physics_engine="Jolt Physics"    ← active Jolt (Partie 3 de la S13)

[application]
run/main_scene="res://scenes/session13_demo.tscn"
```

On **choisit** le moteur physique du projet comme un paramètre. Jolt remplace le moteur par défaut (Godot Physics) depuis la 4.4. Même architecture de cours (broad phase, solveur), autre implémentation — plus rapide, plus stable sur les empilements.

💡 Jolt intégré dans Godot 4.4+

Depuis Godot 4.4, Jolt Physics est disponible comme option de projet : physics/3d/physics_engine = "Jolt Physics". Le projet demo-physics l'utilise. Avantages : stabilité des empilements, performance multithreadée — même architecture, meilleures maths.

Pas de WASM, pas de synchronisation

La différence fondamentale

Contrairement à Rapier, **aucune** étape de chargement n'est nécessaire. La physique démarre avec le moteur. Et contrairement à Three.js + Rapier, **aucune** boucle de synchronisation n'est à écrire : un RigidBody3D est à la fois le corps physique **et** le nœud de scène. Godot met à jour la transformation de toute la hiérarchie quand le moteur avance le body.

Partie 5 : Godot + Jolt — Rigid Bodies & Colliders pas à pas

💡 L'arborescence du projet

```
demo-physics/
├─ project.godot          ← moteur Jolt, scène principale
├─ scenes/
│   └─ session13_demo.tscn ← racine Node3D + le script attaché
├─ scripts/
│   └─ session13_demo.gd   ← tout le code (construit la scène au runtime)
└─ textures/
    └─ grid.png            ← texture du sol (la même que Three.js !)
```

La scène .tscn est minimale : une racine Node3D avec le script attaché. Tout le reste (caméra, lumière, sol, pyramide) est construit **par code** dans _ready(), pour tenir dans un seul fichier comparable à la version Rapier.

Étape 1 — _ready() : construire la scène par code

La méthode d'initialisation

```
extends Node3D

var _cannon_speed := 60.0
var _ccd_enabled := true
```

```

var _camera: Camera3D
var _hud: Label
var _dynamic_bodies: Array[RigidBody3D] = []

func _ready() -> void:
    _setup_camera()
    _setup_light()
    _setup_ground()
    _setup_hud()
    _spawn_stack()

```

`_ready()` est l'équivalent de la `init()` du JS : elle s'exécute une fois au démarrage. Elle appelle cinq fonctions qui créent les sous-systèmes. **Aucune scène n'est préfabriquée dans l'éditeur** — c'est un choix pédagogique pour montrer que Godot permet de tout faire par code.

Étape 2 — Le sol : `StaticBody3D`

Corps FIXE avec enfant `CollisionShape3D` + `MeshInstance3D`

```

func _setup_ground() -> void:
    var ground := StaticBody3D.new()
    ground.position = Vector3(0, -0.1, 0)

    # --- Forme de collision ---
    var col := CollisionShape3D.new()
    var box := BoxShape3D.new()
    box.size = Vector3(60, 0.2, 60)    # ← taille COMPLÈTE
    col.shape = box
    ground.add_child(col)

    # --- Rendu ---
    var mesh := MeshInstance3D.new()
    var box_mesh := BoxMesh.new()
    box_mesh.size = Vector3(60, 0.2, 60)
    mesh.material_override = gridMaterial
    mesh.mesh = box_mesh
    ground.add_child(mesh)

    add_child(ground)

```

Un `StaticBody3D` est le correspondant du `RigidBodyDesc.fixed()` de Rapier. Deux enfants : `CollisionShape3D` (la physique) et `MeshInstance3D` (le rendu). Ils sont **sous le même nœud** : Godot sait qu'il doit synchroniser la forme et le mesh.

Le piège inverse de Rapier

Dans Godot, `BoxShape3D.size` et `BoxMesh.size` prennent la taille **complète** du cube (pas les demi-tailles). C'est plus intuitif, mais il faut le savoir pour ne pas faire un sol de 30 m au lieu de 60 m en transposant naïvement du code Rapier.

Étape 3 — La pyramide : RigidBody3D par code

Chaque cube est un RigidBody3D avec deux enfants

```
func _spawn_stack() -> void:
    var cube_mesh := BoxMesh.new()
    cube_mesh.size = Vector3.ONE

    for row: int in 5:
        var count := 5 - row
        for i in count:
            var cube := RigidBody3D.new()
            cube.position = Vector3((i - (count - 1) / 2.0) * 1.05, 0.5 + row *
1.05, 0)

            var col := CollisionShape3D.new()
            var shape := BoxShape3D.new()
            shape.size = Vector3.ONE
            col.shape = shape
            cube.add_child(col)

            var mesh := MeshInstance3D.new()
            var mat := StandardMaterial3D.new()
            mat.albedo_color = CUBE_COLORS[row % CUBE_COLORS.size()]
            mesh.material_override = mat
            mesh.mesh = cube_mesh
            cube.add_child(mesh)

        add_child(cube)
        _dynamic_bodies.append(cube)
```

Le RigidBody3D est à la fois le corps dynamique et le **conteneur de la hiérarchie**. On lui ajoute un CollisionShape3D pour la physique et un MeshInstance3D pour le rendu. Godot **oriente automatiquement** le mesh quand le body bouge : **aucune boucle de synchronisation** à écrire.

_dynamic_bodies est le pendant du tableau pairs de la version Rapier, mais il ne contient que les corps — le lien avec le mesh est implicite (enfant de chaque body).

Étape 4 — Un corps KINEMATIC (plateforme)

AnimatableBody3D — le kinematic de Godot

```
func _setup_platform() -> void:
    var platform := AnimatableBody3D.new()
    platform.position = Vector3(0, 1.5, -5)

    var col := CollisionShape3D.new()
    var box := BoxShape3D.new()
    box.size = Vector3(6, 0.4, 6)
    col.shape = box
    platform.add_child(col)

    var mesh := MeshInstance3D.new()
    var box_mesh := BoxMesh.new()
    box_mesh.size = Vector3(6, 0.4, 6)
    mesh.material_override = _make_platform_material()
```

```
mesh.mesh = box_mesh
platform.add_child(mesh)
```

```
add_child(platform)
_platform = platform
```

```
# Dans _physics_process : on déplace la plateforme
func _physics_process(delta: float) -> void:
    var t := Time.get_ticks_msec() * 0.001
    _platform.position.x = sin(t) * 4.0
```

AnimatableBody3D (Godot 4) est le successeur du KinematicBody — un corps contrôlé par le code qui pousse les dynamic bodies sans subir les forces. On modifie directement position ; Godot applique le déplacement au prochain pas physique.

💡 AnimatableBody3D vs RigidBody3D en mode kinematic

Un RigidBody3D peut aussi être mis en mode kinematic via `body.mode = RigidBody3D.MODE_KINEMATIC`. AnimatableBody3D est plus léger et conçu pour les **plateformes animées** — il gère le transport des corps qui se trouvent dessus. Pour un ascenseur simple, c'est le bon choix.

Étape 5 — Le tir : rayon caméra + CCD + vitesse

`\_fire\_cannonball()` — le même tunneling, la même démo

```
func _fire_cannonball(screen_pos: Vector2) -> void:
    var from := _camera.project_ray_origin(screen_pos)
    var dir := _camera.project_ray_normal(screen_pos)

    var ball := _make_ball(from + dir * 2.0, 0.45, Color(0.17, 0.24, 0.32), 5.0)
    ball.continuous_cd = _ccd_enabled      # ← CCD ici (propriété du nœud)
    ball.linear_velocity = dir * _cannon_speed # ← vitesse directe
    _ball_count += 1
    _dynamic_bodies.append(ball)
```

La caméra fournit `project_ray_origin()` et `project_ray_normal()` — le raycast écran → monde. On crée un RigidBody3D, on active `continuous_cd`, et on donne une vitesse initiale. Pas d'appel à `apply_impulse` ici : **linear_velocity est équivalent** quand la vitesse initiale est connue.

Étape 6 — Les entrées : _unhandled_input

Un seul callback pour tout

```
func _unhandled_input(event: InputEvent) -> void:
    if event is InputEventMouseButton \
        and event.pressed and event.button_index == MOUSE_BUTTON_LEFT:
        _fire_cannonball(event.position)
    elif event is InputEventKey and event.pressed and not event.echo:
        match event.keycode:
            KEY_R: _reset()
            KEY_C: _ccd_enabled = not _ccd_enabled
```

```
KEY_UP: _cannon_speed = minf(_cannon_speed + 10.0, 150.0)
KEY_DOWN: _cannon_speed = maxf(_cannon_speed - 10.0, 10.0)
```

Un seul gestionnaire remplace les écouteurs de clic + clavier. Godot propose des InputEvent typés. Le match GDScript remplace le switch/if-else.

Étape 7 — Le HUD sleeping

``_process`` lit le moteur à chaque frame

```
func _process(_delta: float) -> void:
    var sleeping := 0
    for body in _dynamic_bodies:
        if body.is_sleeping():
            sleeping += 1
    _hud.text = "Corps dynamiques : %d | Endormis : %d\n" % [ ... ]
    _hud.text += "Vitesse du boulet : %d (↑/↓) | CCD : %s (C)\n" % [ ... ]
```

`_process(delta)` est appelé à **chaque frame rendue** (framerate variable). C'est le bon endroit pour le HUD. Le moteur physique tourne dans `_physics_process(delta)` (60 Hz fixe), mais on n'en a pas besoin ici : Godot gère le `step()` en interne.

Étape 8 — Pas de synchronisation, pas de `step()`

Ce que Godot cache — et pourquoi c'est plus rapide à prototyper

Dans le projet Godot, **aucune boucle** ne copie `body.position` vers `mesh.position`. Le `RigidBody3D` est le parent, le `CollisionShape3D` et le `MeshInstance3D` sont des enfants : Godot met à jour la transformation de toute la hiérarchie quand le moteur avance le body.

Le `world.step()` est lui aussi caché. C'est `_physics_process(delta)` et les paramètres `physics/physics_ticks_per_second` qui font le travail, exactement comme l'accumulateur de la Session 5.

La contrepartie : on a **moins de contrôle** sur le moment exact du pas physique. Pour des jeux classiques c'est un avantage ; pour des réseaux à pas prédictifs ou des simulations très spécifiques, c'est une contrainte.

Partie 6 : Tableau comparatif des API

Concept	Three.js + Rapier (JS)	Godot 4 (GDScript)	Unity 6 (C#, PhysX)
Monde physique	<code>new RAPIER.World(gravity)</code>	Automatique — <code>ProjectSettings / Jolt</code>	Automatique — <code>Physics.scene (PhysX)</code>
Pas fixe	<code>while(accumulator >= FIXED_DT) world.step()</code>	<code>_physics_process(delta)</code> — 60 Hz	<code>FixedUpdate()</code> — 50 Hz par défaut
Chargement	<code>await RAPIER.init()</code> (WASM asynchrone)	Aucune — démarrage immédiat	Aucune — démarrage immédiat
Corps fixe	<code>RigidBodyDesc.fixed()</code>	<code>StaticBody3D</code>	Collider seul (static collider)
Corps dynamique	<code>RigidBodyDesc.dynamic()</code>	<code>RigidBody3D</code>	Rigidbody (défaut)
Corps kinematic	<code>RigidBodyDesc.kinematicPositionBased()</code>	<code>Area3D</code> / <code>Area3D</code>	<code>Rigidbody.isKinematic = true</code>
Forme cubique	<code>ColliderDesc.cuboid(0.5, 0.5, 0.5)</code> (demi-tailles)	<code>BoxShape3D.new(); shape.size = Vector3.ONE</code> (taille complète)	<code>BoxCollider; size = Vector3.one</code> (taille complète)
Forme sphérique	<code>ColliderDesc.ball(0.45)</code>	<code>SphereShape3D.new(); shape.radius = 0.45</code>	<code>SphereCollider; radius = 0.45</code>
Plusieurs colliders	<code>world.createCollider(desc, body) × N</code>	Plusieurs <code>CollisionShape3D</code> enfants	Plusieurs composants Collider
Synchronisation	Manuelle : <code>mesh.position.copy(body.translation())</code>	Automatique : nœud enfant	Automatique : Rigidbody pilote le Transform
CCD	<code>body.setCcdEnabled(true)</code>	<code>ball.continuous_cd = true</code>	<code>rb.collisionDetectionMode = Continuous</code>
Vitesse initiale	<code>body.setLinvel(v)</code>	<code>ball.linear_velocity = v</code>	<code>rb.linearVelocity = v</code>
Impulsion	<code>body.applyImpulse(v, true)</code>	<code>ball.apply_impulse(v)</code>	<code>rb.AddForce(v, ForceMode.Impulse)</code>
Sleeping	<code>body.isSleeping()</code>	<code>body.is_sleeping()</code>	<code>rb.IsSleeping()</code>
Réveil	<code>body.wakeUp()</code>	<code>body.wake_up()</code>	<code>rb.WakeUp()</code>
Gravité	<code>world.gravity = { ... }</code>	<code>ProjectSettings</code> ou <code>Area3D</code>	<code>Physics.gravity</code> ou <code>rb.useGravity</code>
Suppression	<code>world.removeRigidBody(body)</code> + ménage mesh	<code>body.queue_free()</code>	<code>Destroy(gameObject)</code>
Téléportation	À éviter — utiliser forces/impulsions	À éviter — préférer <code>_integrate_forces()</code>	<code>rb.MovePosition()</code> / <code>MoveRotation()</code>

Table 1: Même physique, trois façons de la piloter. Les concepts sont identiques ; seules l'API et l'architecture changent. Unity, comme Godot, intègre son moteur (PhysX) — la synchronisation est automatique.

Partie 7 : Les Colliders en détail

Les formes primitives

Formes disponibles dans les deux moteurs

Les deux moteurs offrent les mêmes primitives — seules les signatures diffèrent :

Forme	Rapier	Godot	Unity
Sphère	<code>ColliderDesc.ball(radius)</code>	<code>SphereShape3D.new(); shape.radius = r</code>	<code>SphereCollider; radius = r</code>
Cuboïde	<code>ColliderDesc.cuboid(hx, hy, hz) (demi)</code>	<code>BoxShape3D.new(); shape.size = v (complet)</code>	<code>BoxCollider; size = v (complet)</code>
Capsule	<code>ColliderDesc.capsule(halfHeight, radius)</code>	<code>CapsuleShape3D.new(); shape.radius / height</code>	<code>CapsuleCollider; radius / height</code>
Cylindre	<code>ColliderDesc.cylinder(halfHeight, radius)</code>	<code>CylinderShape3D.new(); shape.radius / height</code>	<code>MeshCollider (pas de primitive)</code>
Convexe	<code>ColliderDesc.convexHull(points)</code>	<code>ConvexPolygonShape3D.new(); shape.points</code>	<code>MeshCollider; convex = true</code>
Mesh (statique)	<code>ColliderDesc.trimesh(vertices, indices)</code>	<code>ConvexPolygonShape3D.new(); shape.faces</code>	<code>MeshCollider; convex = false</code>

Table 2: Les primitives. Les formes convexes (sphère, cuboïde, capsule, convexe) sont simulées pour les dynamic bodies. Les trimeshes/concaves ne sont que **statiques** — un dynamic body ne peut pas être un mesh concave (le moteur ne sait pas résoudre les collisions sur une forme concave en mouvement). Notons qu'Unity n'a pas de primitive cylindre : on utilise un MeshCollider convexe ou une capsule approchée.

Colliders composés

Plusieurs colliders par body

Un rigid body peut porter **plusieurs** colliders — utile pour les formes complexes (une table = 5 cuboïdes : le plateau + 4 pieds) :

```
// Rapier : on crée plusieurs colliders pour le même body
const table = world.createRigidBody(
  RAPIER.RigidBodyDesc.dynamic().setTranslation(0, 1, 0));
world.createCollider(RAPIER.ColliderDesc.cuboid(1.0, 0.05, 0.6), table); //
plateau
world.createCollider(RAPIER.ColliderDesc.cuboid(0.05, 0.5, 0.05)
  .setTranslation(-0.9, -0.5, -0.5), table); // pied 1
world.createCollider(RAPIER.ColliderDesc.cuboid(0.05, 0.5, 0.05)
  .setTranslation(0.9, -0.5, -0.5), table); // pied 2
// ... pieds 3 et 4

# Godot : on ajoute plusieurs CollisionShape3D enfants
var table := RigidBody3D.new()
table.position = Vector3(0, 1, 0)
```

```

var top := CollisionShape3D.new()
var top_shape := BoxShape3D.new()
top_shape.size = Vector3(2.0, 0.1, 1.2)
top.shape = top_shape
table.add_child(top)

var leg1 := CollisionShape3D.new()
leg1.position = Vector3(-0.9, -0.5, -0.5)
var leg_shape := BoxShape3D.new()
leg_shape.size = Vector3(0.1, 1.0, 0.1)
leg1.shape = leg_shape
table.add_child(leg1)
# ... pieds 2, 3, 4

```

Le moteur calcule le moment d'inertie **global** à partir de tous les colliders — la table tourne de façon réaliste quand on la pousse.

Matériaux : friction & restitution

Les propriétés de surface

Chaque collider porte des propriétés de **matériau** qui correspondent exactement à ce qu'on a codé à la main en Session 6 :

```

// Rapier
RAPIER.ColliderDesc.cuboid(0.5, 0.5, 0.5)
    .setFriction(0.7)           // 0 = glace, 1 = caoutchouc
    .setRestitution(0.3)       // 0 = pas de rebond, 1 = rebond parfait
    .setFrictionCombineRule(RAPIER.CoefficientCombineRule.Average)
    .setRestitutionCombineRule(RAPIER.CoefficientCombineRule.Max);

# Godot — via un PhysicsMaterial
var mat := PhysicsMaterial.new()
mat.friction = 0.7
mat.bounce = 0.3
col.material = mat

```

Les **règles de combinaison** (combine rules) déterminent comment les propriétés de deux colliders en contact se combinent : Average (moyenne), Min, Max, Multiply. Par défaut : friction = moyenne, restitution = max. C'est pourquoi une balle en caoutchouc (restitution 0.9) rebondit haut même sur un sol dur (restitution 0.1) — le Max l'emporte.

Synthèse

Ce qu'il faut retenir

- **Deux philosophies** : Rapier est une **bibliothèque** qu'on intègre à Three.js (synchronisation manuelle) ; Godot **intègre** Jolt (synchronisation automatique via la hiérarchie de nœuds).
- **Le setup** : Rapier nécessite un importmap + await RAPIER.init() (WASM asynchrone) ; Godot nécessite une ligne dans project.godot pour activer Jolt.
- **Le pattern d'intégration** est universel : créer le monde, créer mesh + body par objet, step(), synchroniser. En Rapier on l'écrit ; en Godot il est implicite.

- **Les trois types de corps** (fixed, dynamic, kinematic) existent dans les deux moteurs avec des noms différents mais un comportement identique.
- **Les colliders** sont des objets **séparés** du rigid body en Rapier (attachés après création), et des **nœuds enfants** en Godot. Les formes primitives sont les mêmes ; attention au piège des demi-tailles (Rapier) vs tailles complètes (Godot).
- **Le CCD, le sleeping, les matériaux** (friction, restitution) sont des propriétés uniques à configurer — les concepts de la Session 13 appliqués en une ligne.

💡 La suite

Session 15 : les **joints** — charnières, ressorts, moteurs, ragdolls. On réutilisera les rigid bodies d'aujourd'hui pour les relier entre eux. Puis contraintes et IK (S16), et le TP véhicules (S17) — où tout ce qu'on voit aujourd'hui servira en pratique.